

Praktikum Pengolahan Citra Digital Convolutional Neural Network - CNN

Program Studi Ilmu Komputer

Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami tahapan fundamental CNN pada citra digital
2. Memahami peran convolution, activation, pooling, flatten, dan fully connected layer
3. Mengimplementasikan CNN sederhana menggunakan Python dan library deep learning
4. Melakukan analisis hasil klasifikasi citra pada use case sederhana

Struktur Waktu Praktikum (2 Jam)

- **Bagian 1 – Konsep Fundamental CNN:** 55 menit
- **Bagian 2 – Analisis Use Case Klasifikasi Citra:** 55 menit
- **Diskusi dan refleksi:** 10 menit

1 Pendahuluan

Convolutional Neural Network (CNN) adalah salah satu arsitektur deep learning yang sangat efektif untuk pengolahan citra digital. CNN dirancang untuk mengekstraksi fitur spasial secara otomatis dari citra, mulai dari fitur sederhana seperti tepi dan tekstur, hingga fitur kompleks seperti bentuk objek.

Pada citra digital, setiap piksel menyimpan informasi intensitas atau warna. CNN bekerja dengan mempelajari pola hubungan antar piksel menggunakan operasi konvolusi. Karena itu, CNN menjadi sangat populer untuk tugas seperti:

- klasifikasi citra,
- deteksi objek,
- segmentasi citra,
- pengenalan wajah,
- analisis citra medis dan pertanian.

2 Bagian 1: Konsep Fundamental CNN

2.1 Mengapa CNN untuk Citra?

Jika citra langsung dimasukkan ke neural network biasa (fully connected), jumlah parameter akan sangat besar. Misalnya, citra RGB ukuran $128 \times 128 \times 3$ memiliki:

$$128 \times 128 \times 3 = 49152$$

fitur input. Jika semua piksel langsung dihubungkan ke layer berikutnya, parameter menjadi sangat banyak dan pelatihan menjadi tidak efisien.

CNN mengatasi masalah tersebut dengan:

- memanfaatkan **local connectivity**,
- menggunakan **shared weights** melalui kernel/filter,
- mengekstraksi fitur secara bertahap dari level rendah ke level tinggi.

2.2 Tahapan Utama CNN pada Citra

Secara umum, alur CNN dapat dijelaskan sebagai berikut:

Input Image $\rightarrow$ Convolution $\rightarrow$ Activation $\rightarrow$ Pooling $\rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Output

2.3 Tahap 1: Input Citra

Citra digital dimasukkan ke dalam model dalam bentuk tensor. Misalnya:

- citra grayscale: $H \times W \times 1$
- citra RGB: $H \times W \times 3$

Contoh:

$$128 \times 128 \times 3$$

berarti citra berukuran 128 piksel $\times$ 128 piksel dengan 3 channel warna.

2.4 Tahap 2: Convolution Layer

Convolution adalah proses menggeser filter/kernel kecil di atas citra untuk menangkap pola lokal.

Secara matematis:

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

dengan:

- I = citra input
- K = kernel/filter

- S = feature map hasil konvolusi

Filter awal dapat mendeteksi:

- tepi horizontal,
- tepi vertikal,
- tekstur sederhana,
- sudut atau pola lokal.

Dalam CNN, filter tidak ditentukan manual, tetapi **dipelajari otomatis** selama proses training.

2.5 Tahap 3: Activation Function

Setelah konvolusi, hasilnya dilewatkan ke fungsi aktivasi agar model dapat mempelajari pola non-linear.

Fungsi yang paling umum digunakan adalah ReLU:

$$f(x) = \max(0, x)$$

Makna ReLU:

- nilai negatif diubah menjadi 0,
- nilai positif dipertahankan.

Hal ini membantu model fokus pada aktivasi fitur yang penting.

2.6 Tahap 4: Pooling Layer

Pooling digunakan untuk mereduksi ukuran feature map agar:

- komputasi lebih ringan,
- jumlah parameter berkurang,
- model lebih tahan terhadap pergeseran kecil pada citra.

Pooling yang paling umum adalah max pooling.

Contoh max pooling 2×2 :

$$\begin{bmatrix} 1 & 5 \\ 2 & 4 \end{bmatrix} \rightarrow 5$$

Artinya, hanya nilai maksimum yang diambil dari blok tersebut.

2.7 Tahap 5: Flatten

Feature map hasil convolution dan pooling masih berbentuk matriks/tensor. Agar bisa masuk ke dense layer, tensor tersebut diubah menjadi vektor 1D.

Misal:

$$8 \times 8 \times 32 \rightarrow 2048$$

2.8 Tahap 6: Fully Connected Layer

Setelah flatten, data masuk ke dense layer untuk melakukan proses klasifikasi berdasarkan fitur yang telah dipelajari.

Di tahap ini, model mulai “mengambil keputusan” berdasarkan representasi fitur dari tahap convolution sebelumnya.

2.9 Tahap 7: Output Layer

Jika klasifikasi biner, biasanya digunakan:

- 1 neuron output
- aktivasi sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Jika klasifikasi multi-kelas, biasanya digunakan:

- sejumlah neuron sesuai jumlah kelas
- aktivasi softmax

2.10 Ringkasan Intuisi CNN

CNN belajar secara bertahap:

- layer awal: tepi, garis, tekstur
- layer tengah: bentuk sederhana
- layer dalam: pola objek yang lebih kompleks

Jadi, CNN dapat dianggap sebagai sistem ekstraksi fitur otomatis yang sekaligus melakukan klasifikasi.

3 Implementasi Fundamental CNN dengan Python

Pada bagian ini digunakan TensorFlow/Keras, NumPy, dan Matplotlib.

3.1 Import Library

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

3.2 Membangun Model CNN Sederhana

```
model = models.Sequential([
    layers.Conv2D(16, (3,3), activation='relu',
        input_shape=(128,128,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.summary()
```

3.3 Analisis Arsitektur

Mahasiswa diminta memperhatikan:

- jumlah filter bertambah dari $16 \rightarrow 32 \rightarrow 64$,
- ukuran spasial feature map makin kecil karena pooling,
- semakin dalam layer, semakin kompleks fitur yang dipelajari.

3.4 Pertanyaan Fundamental

1. Mengapa CNN lebih cocok untuk citra dibanding neural network biasa?
2. Apa fungsi kernel/filter dalam convolution?
3. Mengapa ReLU penting dalam CNN?
4. Apa manfaat pooling?
5. Mengapa output flatten diperlukan sebelum dense layer?
6. Apa perbedaan sigmoid dan softmax?
7. Mengapa jumlah filter biasanya bertambah pada layer yang lebih dalam?

4 Bagian 2: Analisis Use Case Klasifikasi Citra

4.1 Use Case

Pada sesi analisis, praktikum menggunakan use case klasifikasi citra sederhana:

Klasifikasi dua kelas citra: Cat vs Dog

Dataset dapat diambil dari Kaggle, misalnya dataset sederhana kategori kucing dan anjing. Dataset semacam ini dipilih karena:

- mudah dipahami,
- label sederhana,
- cocok untuk memperkenalkan alur klasifikasi citra.

4.2 Struktur Folder Dataset

Struktur dataset yang digunakan:

```
dataset/  
train/  
cats/  
dogs/  
val/  
cats/  
dogs/
```

4.3 Tahapan Analisis

Tahapan klasifikasi citra dalam praktikum ini:

1. memahami data dan label,
2. preprocessing citra,
3. membangun model CNN,
4. training model,
5. evaluasi hasil,
6. analisis kesalahan prediksi.

4.4 Load Dataset dengan Library

```
img_size = (128, 128)  
batch_size = 32  
  
train_dataset = tf.keras.utils.  
    image_dataset_from_directory(  
    "dataset/train",  
    image_size=img_size,  
    batch_size=batch_size  
    )
```

```

val_dataset = tf.keras.utils.
    image_dataset_from_directory(
"dataset/val",
image_size=img_size,
batch_size=batch_size
)

class_names = train_dataset.class_names
print(class_names)

```

4.5 Visualisasi Sampel Data

```

plt.figure(figsize=(8,8))
for images, labels in train_dataset.take(1):
for i in range(9):
ax = plt.subplot(3,3,i+1)
plt.imshow(images[i].numpy().astype("uint8"))
plt.title(class_names[labels[i]])
plt.axis("off")
plt.show()

```

4.6 Normalisasi Data

```

normalization_layer = layers.Rescaling(1./255)

train_dataset = train_dataset.map(lambda x, y: (
    normalization_layer(x), y))
val_dataset = val_dataset.map(lambda x, y: (
    normalization_layer(x), y))

```

4.7 Model CNN untuk Use Case

```

model = models.Sequential([
layers.Conv2D(16, (3,3), activation='relu',
    input_shape=(128,128,3)),
layers.MaxPooling2D((2,2)),

layers.Conv2D(32, (3,3), activation='relu'),
layers.MaxPooling2D((2,2)),

layers.Conv2D(64, (3,3), activation='relu'),
layers.MaxPooling2D((2,2)),

layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(1, activation='sigmoid')
])

```

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)  
  
model.summary()
```

4.8 Training Model

```
history = model.fit(  
    train_dataset,  
    validation_data=val_dataset,  
    epochs=5  
)
```

4.9 Visualisasi Akurasi dan Loss

```
acc = history.history['accuracy']  
val_acc = history.history['val_accuracy']  
loss = history.history['loss']  
val_loss = history.history['val_loss']  
  
plt.figure(figsize=(12,5))  
  
plt.subplot(1,2,1)  
plt.plot(acc, label='Train Accuracy')  
plt.plot(val_acc, label='Val Accuracy')  
plt.title('Accuracy')  
plt.legend()  
  
plt.subplot(1,2,2)  
plt.plot(loss, label='Train Loss')  
plt.plot(val_loss, label='Val Loss')  
plt.title('Loss')  
plt.legend()  
  
plt.show()
```

4.10 Prediksi pada Beberapa Citra

```
for images, labels in val_dataset.take(1):  
    preds = model.predict(images)  
  
plt.figure(figsize=(10,10))  
for i in range(9):
```



```
ax = plt.subplot(3,3,i+1)
plt.imshow(images[i].numpy())
pred_label = "dog" if preds[i] > 0.5 else "cat"
true_label = class_names[labels[i]]
plt.title(f"Pred: {pred_label}\nTrue: {true_label}
        ")
plt.axis("off")
plt.show()
```

5 Analisis Hasil

Mahasiswa diminta tidak hanya menjalankan kode, tetapi juga menganalisis proses dan hasilnya.

5.1 Hal yang Harus Diamati

- Apakah akurasi training meningkat dari epoch ke epoch?
- Apakah akurasi validasi mengikuti pola training?
- Apakah loss training menurun?
- Apakah terdapat gap besar antara training dan validation?
- Pada citra yang salah klasifikasi, apa kemungkinan penyebabnya?

5.2 Interpretasi Kemungkinan Hasil

Beberapa kemungkinan hasil:

- **Jika akurasi train tinggi dan validasi juga tinggi:** model belajar dengan baik dan mampu generalisasi.
- **Jika akurasi train tinggi tetapi validasi rendah:** kemungkinan terjadi overfitting.
- **Jika keduanya rendah:** model belum cukup belajar, arsitektur terlalu sederhana, atau data kurang baik.

5.3 Pertanyaan Analisis

1. Bagaimana perubahan akurasi dan loss selama training?
2. Apakah model menunjukkan tanda overfitting? Jelaskan berdasarkan grafik.
3. Mengapa beberapa citra salah diklasifikasikan?
4. Faktor apa yang mungkin memengaruhi performa model? Apakah pencahayaan, sudut objek, latar belakang, atau variasi pose berpengaruh?
5. Jika jumlah epoch ditambah, apa kemungkinan dampaknya?

6. Jika ukuran citra diubah dari 128×128 menjadi 64×64 , apa kemungkinan konsekuensinya?
7. Mengapa klasifikasi citra nyata sering lebih sulit daripada contoh dataset yang sederhana?

5.4 Latihan Pengembangan

Mahasiswa dapat mencoba modifikasi berikut:

- menambah jumlah epoch,
- menambah jumlah convolution layer,
- mengganti jumlah filter,
- menambahkan dropout,
- menggunakan data augmentation.

Contoh data augmentation:

```
data_augmentation = models.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])
```

Pertanyaan lanjutan:

1. Mengapa data augmentation dapat membantu?
2. Dalam kondisi apa dropout diperlukan?
3. Apakah model yang lebih kompleks selalu lebih baik?

6 Refleksi Konseptual

Dari praktikum ini, mahasiswa perlu memahami bahwa CNN bukan sekadar alat klasifikasi, tetapi mekanisme pembelajaran fitur otomatis pada citra. Model tidak diberi tahu secara eksplisit mana tepi, mana tekstur, atau mana bentuk telinga kucing. Semua itu dipelajari dari data melalui proses training.

Dengan demikian, keberhasilan CNN dipengaruhi oleh:

- kualitas data,
- jumlah dan variasi data,
- desain arsitektur,
- parameter training,
- evaluasi dan interpretasi hasil.

7 Kesimpulan

Pada praktikum ini, mahasiswa mempelajari dua hal utama:

1. **Konsep fundamental CNN**, yaitu tahapan input, convolution, activation, pooling, flatten, dense, dan output.
2. **Analisis use case klasifikasi citra**, yaitu bagaimana CNN diterapkan untuk menyelesaikan masalah klasifikasi citra sederhana menggunakan dataset Kaggle.

Praktikum ini diharapkan menjadi dasar bagi mahasiswa untuk memahami pengembangan model CNN yang lebih lanjut, baik untuk klasifikasi multi-kelas, deteksi objek, maupun aplikasi computer vision lainnya.

Catatan untuk Praktikum

Library yang diperlukan:

- tensorflow
- matplotlib
- numpy

Contoh instalasi:

```
pip install tensorflow matplotlib numpy
```